

Arquitectura de datos e IoT

Telemetría multi-dominio, buffer resiliente y detección de personas

Guardian Eye

TFG - Ingeniería de Tecnologías de Telecomunicación, UC3M

Nerea Gorostidi Garcia

1. Propósito del documento

Este documento describe la arquitectura de adquisición y transporte de datos del sistema Drone-SAR tras su ampliación. El objetivo es dejar el modelo cerrado y justificado antes de escribir código, de modo que cada pieza del nodo edge, del transporte por MQTT y del lado servidor tenga una razón de ser explícita y trazable.

La versión anterior del sistema capturaba un único dominio de datos (el sensor ambiental BME680) y lo enviaba a InfluxDB a través de un buffer local. La ampliación persigue tres cosas: incorporar nuevos dominios de datos (estado de vuelo del Pixhawk y estado del propio sistema de la Raspberry Pi), añadir la detección de personas mediante visión por computador, y hacerlo todo sin renunciar a la resiliencia ante cortes de red que ya se tenía.

Cada sección explica no solo qué se hace, sino por qué se ha decidido así, incluyendo las alternativas que se descartaron y el motivo.

2. Visión general de la arquitectura

El sistema se organiza en tres planos: el nodo edge a bordo del dron, el servidor en la nube y la capa de visualización. El nodo edge captura los datos y los transporta; el servidor los recibe, los almacena y los expone; la capa de visualización los consulta.

La idea central de la ampliación es organizar los datos por dominios. Un dominio es un tipo de información con su propio significado y su propio ritmo: los datos de vuelo, los del sistema, los ambientales y las detecciones. Cada dominio viaja por su propio canal y acaba en su propio contenedor dentro de la base de datos, lo que mantiene el sistema ordenado y fácil de ampliar.

Figura 1. Vista global del sistema, del nodo edge a la nube, organizada por dominios de datos.

El nodo edge ejecuta varios procesos independientes, uno por cada dominio de datos. Todos publican por MQTT hacia el broker Mosquitto del servidor EC2, donde un puente traduce los mensajes a InfluxDB y una API los pone a disposición del panel web. Los apartados siguientes desglosan cada plano.

3. Principios de diseño

Tres principios guían toda la arquitectura y conviene tenerlos presentes porque justifican la mayoría de las decisiones posteriores.

3.1. Edge-first

El nodo opera de forma autónoma y la comunicación con la estación base es oportunista, no obligatoria. El dron no depende de tener conexión para hacer su trabajo: captura y registra siempre, y envía cuando puede. Esto es esencial en búsqueda y rescate, donde la cobertura de red es incierta.

3.2. Store-and-forward (almacenar y reenviar)

La captura y el envío están desacoplados. Cada medida se guarda primero en una base de datos local en disco, que actúa como fuente de verdad, y un reenviador la envía más tarde cuando hay conexión confirmada. Si la red se cae, los datos se siguen recogiendo en la Pi y se transmiten al recuperarse el enlace, sin perder ningún intervalo.

Este principio ya existía para el sensor ambiental y se conserva íntegro. La ampliación lo extiende a los nuevos dominios, incluidas las detecciones, que son precisamente el dato que menos nos podemos permitir perder.

3.3. Productor-consumidor

Dentro de cada proceso, quien captura los datos (el productor) y quien los envía (el consumidor) no se comunican directamente, sino a través del buffer local. El productor escribe filas; el consumidor las lee y las publica. El buffer es el punto de encuentro, y ese desacoplamiento es lo que permite que la captura no se detenga aunque falle el envío.

4. El nodo edge en detalle

El nodo edge es una Raspberry Pi 5 con un acelerador Hailo-8L. Sobre ella se ejecutan varios procesos independientes, uno por cada dominio de datos. Cada proceso es autocontenido: captura sus datos, los guarda en su propia base de datos SQLite y los reenvía por MQTT. Esta organización es una de las decisiones de diseño más importantes y merece explicarse con calma.

Figura 2. Nodo edge: un proceso independiente por dominio, cada uno con su propia base de datos SQLite.

4.1. Un proceso por dominio

Cada dominio corre en su propio proceso, gestionado como un servicio de systemd independiente y con su propia base de datos:

- `ambiental.py` lee el BME680 y publica en `sar/dron01/ambiental`.
- `sistema.py` lee el estado de la Pi y publica en `sar/dron01/sistema`.
- `vuelo.py` lee el Pixhawk y publica en `sar/dron01/vuelo`.
- `deteccion.py` ejecuta YOLO y publica en `sar/dron01/deteccion`.

La ventaja de fondo es que este patrón ya existe y está probado: es exactamente el que sigue el `sensor.py` actual, que captura, guarda en su SQLite y reenvía, todo en un mismo proceso. Cada dominio nuevo no es rehacer nada, sino replicar esa plantilla cambiando la parte de lectura y el JSON que se arma. El sensor ambiental, de hecho, ya está resuelto: es `ambiental.py`, y conserva su tabla y su esquema tal cual.

De esta organización se derivan cuatro beneficios directos, y cada uno responde a un problema real:

- Sin bloqueos. Cada proceso es el dueño único de su fichero SQLite: lo escribe y lo lee solo él, nadie más lo toca. Al no haber concurrencia, el problema de los bloqueos desaparece por completo, y ni siquiera hace falta activar el modo WAL.
- Sin planificador. Como cada proceso va a lo suyo, duerme a su propio ritmo con un simple `sleep`: `vuelo` cada 0,3 s, `ambiental` cada 30 s. No hace falta ningún bucle que reparta tiempos entre colectores.
- Aislamiento. Si un proceso falla —por ejemplo, se cuelga la lectura del Pixhawk—, los demás siguen funcionando sin enterarse.
- Progreso escalonado. El desarrollo avanza dominio a dominio: `ambiental` ya funciona, luego se copia para `sistema`, luego para `vuelo` y por último para `detección`, probando cada servicio por separado.

El reenvío de cada proceso conserva la lógica resiliente que ya se tenía: se publica con QoS 1 y una

fila solo se marca como enviada (enviado=1) tras recibir el ACK del broker. Las filas confirmadas se marcan pero, en la implementación actual, no se borran: cada buffer conserva su histórico.

4.2. Reaprovechamiento del código

Los cuatro procesos comparten exactamente el mismo bloque de buffer local y reenvío por MQTT; lo único que cambia entre ellos es qué leen y qué JSON montan. Para no repetir ese bloque cuatro veces, conviene sacarlo a un módulo común —por ejemplo `buffer_mqtt.py`— que los cuatro importen. Así la lógica resiliente vive en un solo sitio y cada colector queda reducido a su lectura y su JSON.

Copiar y pegar el bloque en cada fichero también funcionaría para el alcance del proyecto, pero tiene un coste: el día que haya que tocar el reenvío, habría que corregirlo en cuatro sitios en lugar de en uno. Es una mejora opcional, pero recomendable.

4.3. Configuración desde el entorno: `dron_id` y `client_id`

El identificador del dron no se escribe en el código, sino que se lee de un fichero de entorno `.env`, igual que `sensor.py` ya hace con el resto de su configuración mediante `load_dotenv()`. El `.env` de cada Raspberry Pi contiene su identificador, y el topic se construye a partir de él:

```
# .env (en cada Raspberry Pi) DRON_ID=dron01 EC2_HOST=51.48.220.143 MQTT_PORT=1883
```

```
DRON_ID = os.getenv("DRON_ID")      # p. ej. "dron01" TOPIC = f"sar/{DRON_ID}/ambiental" # el
topic sale del .env client = mqtt.Client( client_id=f"{DRON_ID}-ambiental", # id UNICO por proceso
callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
```

La ventaja es que el mismo código sirve para cualquier dron: para desplegarlo en una segunda unidad basta con cambiar el `DRON_ID` de su `.env`, sin tocar ni una línea. Además, mantiene la coherencia con el criterio que ya se seguía: el código es público y versionable, y todo lo que depende del despliegue vive en el `.env`.

Hay un detalle de MQTT que conviene vigilar: cada proceso necesita un `client_id` único en el broker. Si dos clientes se conectan con el mismo identificador, Mosquitto expulsa a uno y a otro en un bucle sin fin. Por eso el `client_id` se construye combinando el identificador del dron con el nombre del dominio (`dron01-ambiental`, `dron01-vuelo`, etc.): así es único por proceso y, de paso, resulta fácil de reconocer en los logs del broker.

4.4. El proceso de detección y el fichero de estado

El proceso de detección sigue el mismo patrón que los demás, con una diferencia: su «captura» no es una lectura puntual, sino la inferencia de YOLO sobre el vídeo, una carga continua y pesada que corre sobre el Hailo. Tiene su propia base de datos y publica sus alertas en `sar/dron01/deteccion` con la misma lógica de store-and-forward, de modo que una detección tampoco se pierde si la red está caída. Para no repetir avisos de la misma persona en fotogramas seguidos, aplica un control de frecuencia (throttling).

Aparece aquí la única necesidad de comunicación entre procesos: para que una detección diga dónde está la persona, el proceso de detección necesita la posición del dron en ese instante, pero esa posición la lee el proceso de vuelo. La solución es un fichero de estado. Cada vez que el proceso de vuelo lee el Pixhawk, sobrescribe un fichero pequeño con la última posición conocida:

```
// posicion_actual.json { "lat": 40.6031, "lon": -4.0018, "alt_rel": 45.2, "ts":  
"2026-08-10T12:34:56.789+02:00" }
```

Un solo proceso escribe ese fichero (el de vuelo) y un solo proceso lo lee (el de detección, y solo cuando detecta algo). No es una base de datos, es un fichero diminuto de estado, así que no reabre el problema de los bloqueos que precisamente se quería evitar. Es la única pieza compartida entre procesos.

Dos detalles lo hacen robusto. Primero, la escritura es atómica: el proceso de vuelo escribe a un fichero temporal y luego lo renombra, de modo que el de detección nunca lea un fichero a medio escribir. Segundo, el JSON lleva su propia marca de tiempo (ts), así que se puede comprobar si la posición está fresca; si fuese de hace varios segundos, se puede señalar en el mensaje en lugar de dar por buena una posición vieja.

5. Nomenclatura de topics MQTT

El esquema de topics anterior era plano (por ejemplo, un topic por dominio bajo una raíz común) y no distinguía de qué dron procedía cada mensaje. En cuanto se contempla más de un dron —o simplemente para dejarlo preparado— ese esquema se queda corto. Se adopta una convención jerárquica, de lo general a lo específico:

```
sar / {dron_id} / {dominio}
```

Figura 3. Estructura jerárquica de topics y los comodines que habilita en el lado servidor.

El primer nivel es la raíz del proyecto; el segundo identifica el dron que emite; el tercero indica el dominio de datos. Cada dron cuelga de su propio subárbol, y dentro de él aparecen sus cuatro dominios: vuelo, sistema, ambiental y deteccion.

5.1. Reglas de nomenclatura

Para que la convención sea sólida y no dé problemas, los nombres siguen unas reglas estrictas:

- Solo minúsculas y ASCII: sin tildes ni ñes. Por eso deteccion va sin acento.
- Sin espacios, y sin barra al principio ni al final.
- Nunca los caracteres + ni # en los nombres reales, porque están reservados como comodines.
- Estructura estable: el dron_id es fijo. Nunca se meten datos que cambian (como una marca de tiempo) dentro del topic; esos van en el mensaje.

5.2. Comodines: la ventaja de la jerarquía

La estructura jerárquica es lo que permite suscribirse con comodines en el servidor, y ahí está su verdadero valor:

- sar/+/deteccion suscribe a todas las detecciones de todos los drones; es el canal de alertas, el importante.
- sar/dron01/# suscribe a todo lo que emite un dron concreto.
- sar/# suscribe absolutamente a todo.

6. Identificación del dron

Con esta convención, el identificador del dron vive en el topic y no hace falta repetirlo dentro de cada mensaje. El puente del servidor extrae el identificador del propio topic y lo guarda en InfluxDB como

etiqueta (tag), no como campo (field).

Esta distinción importa en InfluxDB: los tags están indexados, mientras que los fields no. Al guardar el identificador como tag, el panel puede filtrar y agrupar por dron de forma eficiente («dame las detecciones donde dron_id es dron01») sin recorrer todos los datos. Guardarlo como field sería mucho menos eficiente para ese tipo de consulta.

Para el identificador se usa algo corto y legible, del tipo dron01. En una flota real se emplearía un número de serie o un UUID, pero para el alcance del proyecto un identificador legible es más claro y suficiente. Como excepción práctica, en el mensaje de detección sí se incluye además el identificador dentro del propio JSON, para que la alerta sea autoexplicativa por sí sola aunque se maneje fuera de su contexto.

Ese identificador es la única fuente de la verdad sobre qué dron es cada Pi, y por eso vive en su fichero de entorno .env (apartado 4.3). De ahí lo leen todos los procesos para construir tanto el topic como su client_id, de forma que un mismo código se despliega en cualquier unidad cambiando solo esa variable.

7. Modelo de datos por dominio

Se definen a continuación los cuatro dominios con su estructura JSON, sus campos y sus unidades. Todos los mensajes llevan una marca de tiempo en formato ISO 8601 con zona horaria, tomada en el instante de la captura y no en el del envío, para que un dato enviado con retraso conserve su instante real.

7.1. Dominio ambiental (BME680)

Topic: sar/{dron_id}/ambiental. Es el dominio que ya existía y se mantiene sin cambios en su contenido.

```
{ "temperatura": 22.45, "humedad": 48.3, "presion": 1013.2, "timestamp":  
"2026-08-10T12:34:56.789+02:00" }
```

Campo	Tipo / unidad	Descripción
temperatura	float · °C	Temperatura, redondeada a 2 decimales.
humedad	float · % HR	Humedad relativa.
presion	float · hPa	Presión atmosférica.
timestamp	texto · ISO 8601	Instante de captura con zona horaria.

7.2. Dominio sistema (Raspberry Pi)

Topic: sar/{dron_id}/sistema. Informa del estado del propio nodo edge. Los valores se obtienen con psutil y vcgencmd.

```
{ "cpu_pct": 34.5, "cpu_temp": 61.2, "ram_pct": 42.0, "disco_pct": 55.3, "throttled": "0x0",  
"uptime_s": 4213, "timestamp": "2026-08-10T12:34:56.789+02:00" }
```

Campo	Tipo / unidad	Descripción
cpu_pct	float · %	Uso de CPU.
cpu_temp	float · °C	Temperatura del procesador.

Campo	Tipo / unidad	Descripción
ram_pct	float · %	Uso de memoria RAM.
disco_pct	float · %	Uso del disco.
throttled	texto · hex	Código de vcgencmd; avisa de bajadas de tensión.
uptime_s	entero · s	Segundos desde el último arranque.
timestamp	texto · ISO 8601	Instante de captura con zona horaria.

El campo throttled es especialmente útil en este proyecto: la Pi 5 y el Hailo se alimentan desde un UBEC sobre batería LiPo, y ese código avisa de caídas de tensión. Es un dato que respalda con evidencia el dimensionado de la alimentación.

7.3. Dominio vuelo (Pixhawk)

Topic: sar/{dron_id}/vuelo. Recoge la telemetría de vuelo leída del Pixhawk por MAVLink. Los campos provienen de mensajes MAVLink concretos: posición (GLOBAL_POSITION_INT), actitud (ATTITUDE), batería (SYS_STATUS), GPS (GPS_RAW_INT) y modo (HEARTBEAT).

```
{ "lat": 40.6031, "lon": -4.0018, "alt_rel": 45.2, "vel_terreno": 5.4, "rumbo": 178, "roll": 1.2, "pitch": -0.8, "yaw": 178.4, "bateria_v": 22.4, "bateria_pct": 78, "corriente_a": 12.3, "satelites": 14, "fix_type": 3, "modo": "AUTO", "armado": true, "timestamp": "2026-08-10T12:34:56.789+02:00" }
```

Campo	Tipo / unidad	Descripción
lat, lon	float · grados	Posición geográfica (grados decimales).
alt_rel	float · m	Altitud relativa al punto de despegue.
vel_terreno	float · m/s	Velocidad respecto al suelo.
rumbo	float · grados	Rumbo (heading).
roll, pitch, yaw	float · grados	Actitud del dron.
bateria_v	float · V	Tensión de la batería.
bateria_pct	entero · %	Carga restante estimada.
corriente_a	float · A	Corriente consumida.
satelites	entero	Número de satélites en uso.
fix_type	entero · 0-6	Calidad del fix GPS.
modo	texto	Modo de vuelo (AUTO, LOITER, etc.).
armado	booleano	Si los motores están armados.
timestamp	texto · ISO 8601	Instante de captura con zona horaria.

7.4. Dominio detección (YOLO)

Topic: sar/{dron_id}/deteccion. Es una alerta que se emite cuando se localiza una persona. El mensaje envía los ingredientes en crudo: la posición del dron da la zona, y los píxeles de la caja indican dónde cae la persona dentro del fotograma.

```
{ "confianza": 0.87, "caja": { "cx": 412, "cy": 230, "w": 48, "h": 96 }, "resolucion": { "ancho": 832, "alto": 464 }, "dron": { "lat": 40.6031, "lon": -4.0018, "alt_rel": 45.2 }, "timestamp": "2026-08-10T12:34:56.789+02:00" }
```

Campo	Tipo / unidad	Descripción
confianza	float · 0-1	Confianza de la detección.
caja	objeto · px	Caja del objeto: centro (cx, cy) y tamaño (w, h).
resolucion	objeto · px	Resolución del fotograma, para interpretar la caja.
dron	objeto	Posición del dron en el instante de la detección.
timestamp	texto · ISO 8601	Instante de la detección con zona horaria.

Se han tomado varias decisiones conscientes en este mensaje. La caja se expresa como centro más tamaño en vez de esquinas: el centro es «dónde está la persona en la imagen», que es lo que interesa, y el tamaño da una idea de si está cerca o lejos. Se incluye la resolución a propósito, porque sin saber que los píxeles son sobre 832×464 no se pueden interpretar aguas abajo; es un dato barato que hace el mensaje autoexplicativo. Si en un fotograma hay varias personas, se emite un mensaje por persona.

Una advertencia importante para no cometer imprecisiones: YOLO no entrega coordenadas geográficas de la persona, sino su posición en píxeles dentro de la imagen. Este mensaje adopta el enfoque más simple y honesto —enviar la posición del dron más los píxeles— que basta para guiar a un equipo de tierra a una zona de búsqueda. Estimar la posición real de la persona sobre el terreno exigiría un proceso de georreferenciación (fusionar píxeles con altitud y actitud), que queda como posible línea futura. Dada la resolución de trabajo (832×464) y las personas pequeñas en cuadro, esa estimación tendría un margen de varios metros, por lo que no debe presentarse como localización exacta.

Nota de coherencia sobre el formato: en el dominio de vuelo, lat/lon/alt_rel van al primer nivel porque son el flujo de telemetría en sí; en la detección van anidados bajo dron porque ahí son una foto puntual embebida dentro de la alerta.

8. El flujo de una detección

Conviene ver el camino completo de una alerta, porque atraviesa las dos mitades del sistema y reúne varias de las decisiones anteriores.

Figura 4. Secuencia completa de una detección de persona, del nodo edge a InfluxDB.

Cuando el proceso de YOLO detecta una persona, lee la posición del dron del fichero de estado, arma el mensaje JSON y lo publica en sar/dron01/deteccion con QoS 1. El broker Mosquitto confirma con un ACK, y solo entonces la fila se marca como enviada en el buffer propio de YOLO. A partir de ahí, el puente del servidor entrega el mensaje a InfluxDB, donde se escribe como un punto etiquetado con el identificador del dron. Si en cualquier punto de la cadena hay un corte de red, el mensaje permanece a salvo en el buffer y se reintenta después.

9. El lado del servidor

El nuevo modelo obliga a un cambio pequeño pero necesario en el puente mqtt-to-influx del servidor EC2. En lugar de suscribirse a un topic fijo, pasa a suscribirse al comodín sar/#, que abarca todos los drones y todos los dominios.

Por cada mensaje que llega, el puente parte el topic en sus componentes (dron_id, dominio), escribe el contenido en la measurement que corresponde al dominio (una por cada tipo de dato) y añade el dron_id como tag. De este modo, un único puente sirve para los cuatro dominios y para cualquier número de drones, sin necesidad de código específico por cada uno.

El resto de la infraestructura del servidor (el broker, la base de datos, la API y el panel) no cambia su papel: el broker recibe, la base de datos almacena por measurements, la API consulta y el panel visualiza.

10. Qué se puede validar sin hardware

Una ventaja de este diseño es que puede construirse y probarse por partes, sin esperar a tener montado el dron ni el modelo desplegado en el Hailo. El desarrollo puede avanzar en paralelo a la compra y el ensamblaje del hardware.

- El flujo de detección (detectar, leer la posición, montar el mensaje y publicar) se prueba ya con Ultralytics sobre un vídeo de vuelo real en CPU y un fichero de posición simulado, validando todo el camino end-to-end sin hardware.
- El colector de vuelo se desarrolla y prueba contra el simulador ArduPilot SITL con pymavlink; cuando llegue el Pixhawk, solo cambia la cadena de conexión.
- Cuando exista el modelo real convertido a Hailo, solo se sustituye el motor de inferencia; el resto del código de detección no se toca.

Este enfoque incremental sigue el mismo criterio de validar el flujo de trabajo antes de comprometer el sistema definitivo que ya se aplicó en la fase preliminar de visión.

11. Resumen de decisiones y su justificación

La siguiente tabla condensa las decisiones de diseño de la ampliación y el motivo de cada una, a modo de referencia rápida.

Decisión	Por qué
Un dominio = un topic = una measurement	Mantiene los datos ordenados por significado y hace trivial añadir dominios nuevos sin rehacer nada.
Topics jerárquicos sar/{dron_id}/{dominio}	Identifica el dron dentro del propio topic y habilita comodines para suscribirse por dron o por dominio.
Identificador del dron como tag en InfluxDB	Los tags están indexados: permiten filtrar y agrupar por dron de forma eficiente.
Un proceso por dominio con su propia SQLite	Cada proceso es dueño único de su base de datos: sin concurrencia ni bloqueos. Replica el patrón ya probado de sensor.py y permite avanzar de forma escalonada.
Cada proceso duerme a su ritmo (sleep propio)	Al ser independientes, no hace falta ningún planificador que reparta tiempos entre colectores.

Decisión	Por qué
Lógica de buffer y reenvío en un módulo común	Evita duplicar el mismo bloque en los cuatro procesos: un único sitio que mantener si cambia el reenvío.
dron_id y client_id leídos del .env	El mismo código sirve para cualquier dron cambiando solo el .env; el client_id único por proceso evita que Mosquito expulse clientes duplicados.
YOLO como un proceso más	Encaja en el mismo patrón; su única particularidad es que su captura es inferencia continua en vez de una lectura puntual.
Fichero de estado para la posición	Comparte la posición entre vuelo y detección sin una base de datos compartida, sin reabrir el problema de los bloqueos.
Escritura atómica (tmp + rename)	Garantiza que el lector nunca vea un fichero de posición a medio escribir.
QoS 1 y marcado tras ACK	Asegura que ningún dato se dé por enviado sin confirmación real del broker.
Detección: dron + píxeles en crudo	Es lo alcanzable y honesto; no inventa una precisión geográfica que el sistema no tiene.

Nota sobre el índice